

GraphPoint: Semantic Entity Graphs and Point Trajectories for Compositional Robot Manipulation

Kang Luo, Hesheng Wang[†]

Abstract—

I. INTRODUCTION

II. RELATED WORK

A. Visual and Point-Based Manipulation Policies

Visual imitation learning provides a practical route from demonstrations to robot control. ACT [1] predicts action chunks for fine-grained manipulation, while Diffusion Policy [2] models action sequences through conditional denoising. At larger scale, $\pi_{0.5}$ [3] combines heterogeneous training data with vision-language-action modeling to support generalization to new environments. These approaches establish strong image-conditioned policies, while 3D representations offer a complementary geometric interface. DP3 [4] conditions diffusion policies on compact features extracted from sparse scene point clouds. PointPolicy [5] represents observations and actions using semantic key points, enabling policy learning from human demonstration videos. More recently, Point Bridge [6] uses unified point representations for synthetic-data training and sim-to-real transfer. GraphPoint builds on this point-based perspective, organizing task-relevant observations by semantic entity roles and predicting actor point trajectories. Our focus is how this organization supports instruction-dependent task composition.

B. Object-Centric and Graph-Based Representations

Object-centric representations expose individual entities as reusable components of a scene. GROOT [7] combines segmented object point clouds with a transformer policy and segmentation correspondence to handle changes in backgrounds, viewpoints, and object instances. Graph-based approaches further represent interactions between entities. GraphMimic [8] abstracts videos into object and visual-action vertices, pretrains a model to generate future graphs, and conditions a control policy on these predictions. Compose by Focus [9] selects task-relevant scene subgraphs, encodes them with a graph attention network, and combines diffusion-based atomic skills with a vision-language task planner. These works establish object structure and task-relevant relations as useful priors for control and skill composition. GraphPoint studies a specific combination of actor/patient/target roles, explicit action-type and modifier conditioning, and point-trajectory generation. Its distinction lies in this representation and policy interface; object-centric

inputs and graph-based skill composition are established ingredients.

C. Compositional Generalization in Manipulation

Compositional generalization concerns new combinations of familiar task factors as well as new sequences of learned behaviors. CLIPort [10] couples pretrained semantic features with spatially precise action prediction for language-conditioned manipulation. VIMA [11] introduces multimodal task prompts and a benchmark with 4 generalization levels, while CompoSuite [12] explicitly combines robots, objects, obstacles, and objectives to evaluate unseen task combinations. LIBERO [13] studies declarative and procedural knowledge transfer across manipulation tasks, and CALVIN [14] evaluates language-conditioned continuous control over multi-step task sequences. These benchmarks address complementary forms of transfer whose difficulty depends on the factors held out during training. CoMani provides a focused LIBERO-based evaluation of modifier combinations at a fixed action type, action-type combinations at a fixed modifier, and unseen sequences of atomic tasks. CoMani-Seq tests execution of fixed 3-step instructions composed of familiar and held-out atomic combinations, while matched initial scenes reduce opportunities to infer the task from visual context alone.

III. METHOD

A. Overview

GraphPoint represents manipulation as interactions among 3D point entities and predicts future gripper points in the same geometric space. As shown in Fig. 1, language analysis, visual localization, and point tracking construct a task-conditioned entity graph, which a shared policy maps to a point trajectory and a subtask progress estimate. For a multi-step instruction, the system instantiates this graph for each subtask and uses predicted progress to advance through the sequence. The learned policy receives entity geometry and structured language conditions; RGB images are used by the perception front end.

B. Task-to-Graph Construction

Semantic decomposition. A language model decomposes an instruction into ordered subtasks, each specified by an action type, a modifier, and entity roles. We distinguish the *actor* A (the gripper), the *patient* P (the manipulated object or part), and the *target* T (the reference entity). For example, “put the cheese to the right of the stove” assigns cheese to P , stove to T , *put* to the action type, and *right* to

Kang Luo and Hesheng Wang are with IRMV Lab, the Department of Automation, Shanghai Jiao Tong University.

[†]Corresponding author email: wanghesheng@sjtu.edu.cn

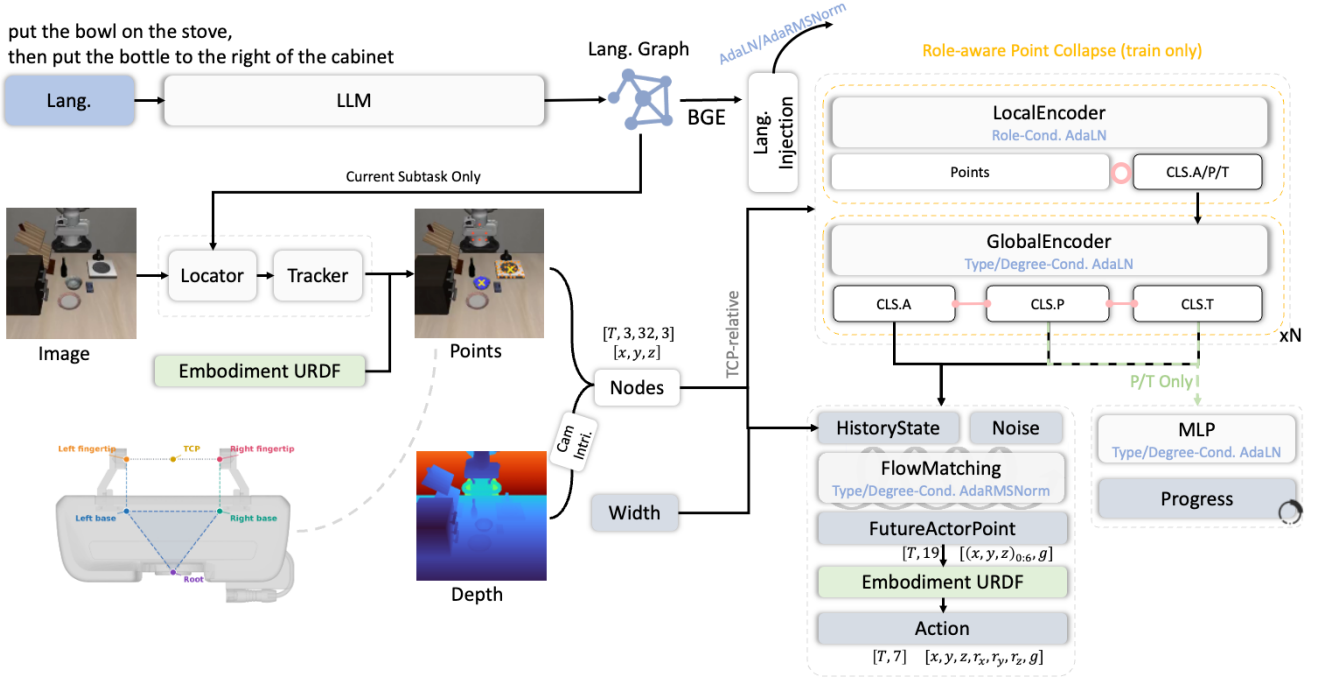


Fig. 1. GraphPoint framework. Language analysis identifies ordered subtasks and their semantic roles, while visual localization and tracking ground the relevant entities as 3D points. The policy alternates local entity encoding with temporal graph attention, conditioned on roles, action types, and modifiers. A flow decoder predicts future actor points and gripper commands, and a progress head supports subtask transitions. Robot geometry converts predicted points into executable gripper poses.

the modifier. Entity names guide visual grounding, whereas action types and modifiers are encoded separately using frozen BGE embeddings [15] and learned projections. We denote their concatenation by the task condition c and the learned embedding of role r by e_r . For readability, c also denotes the version with separately normalized action and modifier embeddings used by the prediction heads. Absent modifiers use a learned null embedding, and absent entities are masked.

Grounded point entities. A pretrained vision-language locator identifies the requested objects or parts, and SAM 2 [16] produces masks from the localization prompts. We sample 32 points per visual entity and track their image coordinates with TAPNext++ [17]. Depth and camera intrinsics back-project these tracks into 3D. The actor instead uses 6 keypoints computed from robot geometry and proprioception: the gripper root, TCP, 2 finger bases, and 2 fingertips. We align all entities in camera coordinates and subtract the current TCP position from every point in the observation history and prediction horizon. Thus, coordinates retain camera-axis orientation while sharing a translation origin at the current TCP. This centering expresses scene geometry and future motion relative to the current gripper location, reducing dependence on absolute scene placement; the uncentered ablation retains the camera-frame origin. Actor points are padded with a validity mask, and coordinates are normalized using training-set quantiles. The policy observes 10 frames, including the current frame.

C. Role-Structured Entity Graph Encoder

Role-conditioned geometry and task-conditioned interaction. A shared point MLP embeds entity coordinates, with keypoint identity embeddings added for the actor. Each entity at each observed time receives 4 learned CLS tokens to summarize its geometry. The encoder alternates local encoding within each entity and global interaction among entity summaries:

$$\begin{aligned} (\tilde{C}_r, F_r^{\ell+1}) &= \text{Local}_\ell(C_r^\ell, F_r^\ell; e_r), \\ \tilde{C}^{\ell+1} &= \text{Graph}_\ell(\{\tilde{C}_r\}_{r \in \{A, P, T\}}; c). \end{aligned} \quad (1)$$

Here, F_r and C_r denote point features and CLS summaries, respectively, with observation-time indices omitted for clarity. Local self-attention processes each entity and frame independently, conditioned on its role e_r . Graph attention exchanges only CLS summaries across entities and observed times, conditioned on the action and modifier c . Its connectivity follows the bidirectional chain $A \leftrightarrow P \leftrightarrow T$, with additional within-role temporal connections. Direct actor-target attention is masked, so their interaction is mediated by the patient through successive layers. The chain encodes gripper-object and object-reference interactions explicitly; full connectivity removes this communication constraint by also allowing direct actor-target attention. Separating entities also preserves distinct role-conditioned geometric summaries, whereas the merged-point-set ablation pools the entities into a single set and removes separate local role groups and role embeddings. These variants distinguish the communication pattern from the broader choice to maintain

a role-structured graph. Temporal rotary embeddings distinguish observation times. After 8 layers, the current-time CLS summaries form memory M_t , incorporating both geometry and interaction history.

Semantic injection. Both local and graph blocks contain attention and feed-forward sublayers, whose inputs are modulated through adaptive layer normalization:

$$\text{AdaLN}(h; q) = (1 + \gamma(q)) \odot \text{LN}(h) + \beta(q). \quad (2)$$

The scale γ and shift β are learned functions of the condition, broadcast over tokens. Using $q = e_r$ locally lets the encoder interpret geometry according to whether it belongs to the actor, patient, or target. Using $q = c$ globally makes the same entity configuration task-dependent, such as distinguishing placing an object *on* a target from placing it to its *right*. Thus, role semantics control entity encoding, while action and modifier semantics control interaction through feature modulation.

Role-aware point collapse. During training, we independently collapse each patient or target point set with probability 0.05 to reduce dependence on detailed object shape. For the patient, all points are replaced by the mean of the 4 points nearest to the TCP; for the target, they are replaced by the entity centroid. This operation is applied consistently over the sampled temporal window, with anchors computed separately at each frame. It preserves point counts and validity masks and leaves actor points and future action supervision unchanged. The augmentation exposes the encoder to simplified geometry while retaining a role-specific spatial anchor. The patient anchor emphasizes geometry near the interaction region, while the target anchor retains a reference location; removing RPC tests the contribution of this geometric augmentation.

D. Point-Trajectory Flow Policy

We predict $H = 10$ future steps, each containing 6 actor points and a scalar gripper command:

$$Y = \{(X_{t+h}^A, g_{t+h})\}_{h=1}^H. \quad (3)$$

Here, X_{t+h}^A contains actor coordinates relative to the current TCP, and g_{t+h} is the open/close command. Each step is represented by a 19-dimensional vector in point-wise XYZ order followed by the gripper command. Representing actions as gripper points makes the predicted motion spatially comparable to the observed entities and encodes orientation through the arrangement of multiple points. The delta-action ablation instead predicts Cartesian translation and rotation increments with a gripper command, testing the choice of action representation.

Geometry- and language-conditioned decoding. A 6-layer Transformer models this trajectory with conditional flow matching [18]. It embeds the observed actor history U_t together with the noisy future trajectory Z_τ and predicts a velocity field:

$$v_\theta = \text{Decoder}_\theta(Z_\tau, U_t; M_t, c, \tau). \quad (4)$$

Each decoder layer uses self-attention to relate trajectory steps, cross-attention to retrieve entity geometry from M_t , and a feed-forward sublayer to update features. Future tokens attend to both history and future, while history tokens cannot attend to future tokens. Temporal rotary embeddings encode their relative times. Action and modifier semantics enter every sublayer through AdaRMSNorm, following the scale-and-shift principle of Eq. (2) with RMS normalization. Separate mappings of the task condition c and flow-time embedding are added to generate the scale, shift, and residual gate. Consequently, geometry guides decoding through cross-attention, while language and flow time modulate how each layer updates the trajectory. A final projection of the future tokens produces the point and gripper velocity predictions.

Training and execution. For a demonstrated trajectory Y and Gaussian noise ϵ , the flow objective is

$$\begin{aligned} Z_\tau &= (1 - \tau)Y + \tau\epsilon, \\ \mathcal{L}_{\text{flow}} &= \mathbb{E}[\text{MSE}(v_\theta, \epsilon - Y)], \end{aligned} \quad (5)$$

where the squared error is averaged over trajectory steps and output dimensions. At inference, 10 Euler steps integrate from noise at $\tau = 1$ to a trajectory at $\tau = 0$. After denormalization and restoring the TCP translation, we fit the robot’s gripper keypoint template to each predicted point set using least-squares rigid alignment. The template uses the observed gripper width, and the predicted gripper command is applied separately. The recovered poses are transformed into the control frame and executed in a receding-horizon manner. This geometric conversion provides the interface between point predictions and the robot controller.

E. Subtask Progress and Long-Horizon Composition

The progress head estimates how far the active subtask has advanced from the current patient and target summaries:

$$\hat{s}_t = \text{Progress}_\theta(C_{t,P}, C_{t,T}; c). \quad (6)$$

It uses a task-conditioned MLP with AdaLN and a sigmoid output, so the same geometry is evaluated under the active action and modifier. Using patient and target summaries focuses the readout on the manipulated entity and its relation to the reference, rather than directly including gripper summaries as progress cues. This restriction applies to the readout: patient and target summaries can still contain actor information propagated through graph attention. The A+P+T ablation adds actor summaries to the standard P+T inputs, testing whether directly exposing gripper state helps or hinders progress-based switching. Training targets increase linearly from 0 to 1 within each annotated subtask interval, and the complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \text{SmoothL1}(\hat{s}_t, s_t). \quad (7)$$

At deployment, the planner advances when predicted progress exceeds a threshold for a consecutive window of policy calls. We denote this learned transition rule as self-switching (SS); environment-based switching (ES) instead uses an environment-provided subtask completion signal to separate execution performance from learned transition

timing. It performs a release-and-lift transition, selects the next subtask’s entities and language conditions, and resets policy history. The same policy is reused for every subtask, enabling execution of task sequences assembled from atomic demonstrations without requiring a separate policy for each sequence.

IV. CoMANI BENCHMARK

CoMani comprises 3 suites testing modifier, action-type, and sequential composition (Fig. 2; Table I). We generate 50 trajectories per atomic task using MimicGen [19]. Within each suite, initial scenes are matched wherever feasible to require instruction-dependent behavior and discourage fixed vision-to-action mappings. We report success rates by split over 30 trials per atomic task and 10 per sequence.

CoMani-Mod. With pick-and-place (*put*) fixed, this suite combines 3 objects, 3 targets, and 2 modifiers (*on/right of*) into 18 tasks: 12 in-distribution (ID) and 6 out-of-distribution (OOD). Each OOD object–target pair appears in the ID split with the opposite modifier, isolating generalization to unseen combinations of familiar factors. The ID/OOD comparison therefore tests whether a policy can apply the instructed relation to a familiar object–target pair beyond its demonstrated placement.

CoMani-Act. With *right* fixed, this suite varies *put*, *sweep*, and *rotate* across 10 tasks: 7 ID and 3 OOD. For *put/sweep*, each OOD object–target pair appears in the ID split with the other action; for rotation, the ID task uses the stove knob and the OOD task uses cream cheese. These tests examine whether a policy can reuse an action with familiar entities in a new combination and transfer rotation across different manipulated entities. *Put/sweep* pairs share initial states and final-position goals, so success measures goal attainment rather than interaction mode.

CoMani-Seq. This suite tests whether policies learned from CoMani-Mod atomic demonstrations can execute 3-step instructions without sequence-level demonstrations. It contains 6 sequences organized into 3 fixed-order pairs, each varying one modifier between *on* and *right of*, without permuting subtasks. The 3 pairs contain 3 ID, 2 ID + 1 OOD, and 2/1 ID + 1/2 OOD atomic tasks, respectively (Fig. 2, right). The all-ID pair tests composition and transitions among familiar skills; the remaining pairs test whether execution also generalizes to held-out atomic combinations, with the last pair increasing the OOD count from 1 to 2 through a single modifier change.

V. EXPERIMENTS

We evaluate GraphPoint on CoMani-Mod, CoMani-Act, and CoMani-Seq using the task splits and protocols in Sec. IV. We report success rates separately for ID and OOD tasks over 30 trials per atomic task and 10 trials per sequence. For the reported atomic-task experiments, GraphPoint is trained with 10 demonstrations per ID task and is evaluated at 30,000 training steps. Tables report mean task success rates; blank entries indicate evaluations that are not yet complete. Our experiments address 4 questions:

- Q1:** Can GraphPoint generalize across modifiers?
- Q2:** Can GraphPoint generalize across actions?
- Q3:** Can GraphPoint compose atomic skills?
- Q4:** Which components matter?

We answer the first 3 questions through comparisons on the corresponding suites and the fourth through ablations on CoMani-Mod.

A. Can GraphPoint Generalize Across Modifiers?

We compare GraphPoint with ACT [1], Diffusion Policy (DP) [2], DP3 [4], PointPolicy [5], $\pi_{0.5}$ [3], and PointBridge [6]; the ID evaluation of PointBridge remains in progress. As shown in Table II, GraphPoint achieves 96.39% ID and 75.56% OOD success, exceeding the highest baseline ID success ($\pi_{0.5}$, 91.11%) by 5.3 percentage points and the highest baseline OOD success (DP3, 57.22%) by 18.3 points. $\pi_{0.5}$ obtains 48.33% OOD success. ACT, DP, and PointPolicy also have low ID success in this setting, so their OOD results reflect limitations in both task acquisition and generalization. The comparisons with DP3 and $\pi_{0.5}$ provide references with stronger ID performance for assessing held-out modifier combinations.

GraphPoint achieves at least 63.33% success on every OOD task, whereas DP3 ranges from 16.67% to 93.33%. Its gains are particularly large for placing the wine bottle on the cabinet (73.33% versus 16.67%) and to the right of the plate (66.67% versus 23.33%). However, DP3 performs better on placing cream cheese to the right of the cabinet (93.33% versus 76.67%). These results support broader coverage of held-out relations by GraphPoint, while the remaining ID–OOD gap shows that modifier composition is still challenging.

B. Can GraphPoint Generalize Across Actions?

Table III compares GraphPoint with DP3 on CoMani-Act, with $\pi_{0.5}$ left pending. GraphPoint reaches 90.48% ID and 64.44% OOD success, compared with 83.33% and 60.00% for DP3. The modest average OOD advantage masks substantial differences across held-out tasks. GraphPoint transfers rotation from the stove knob to cream cheese with 76.67% success, while DP3 obtains 0%. Conversely, GraphPoint trails DP3 on putting cream cheese to the right of the stove (26.67% versus 80.00%) and sweeping the bowl to the right of the plate (90.00% versus 100.00%). Thus, these results support cross-entity rotation transfer but do not establish a uniform advantage across action combinations. For *put/sweep*, the reported metric verifies the final spatial goal and does not independently verify the interaction mode.

C. Can GraphPoint Compose Atomic Skills?

We compare GraphPoint with DP3 and $\pi_{0.5}$ on CoMani-Seq to assess execution of 3-step instructions using policies trained on atomic-task demonstrations, without sequence-level demonstrations. Table IV reports success rate (SR) and progress rate (PR) over 10 trials per sequence, together with their means across all 6 sequences. Tasks 0–1 test composition of 3 ID skills, tasks 2–3 introduce 1 OOD atomic task, and tasks 4–5 contain 1 and 2 OOD atomic tasks,

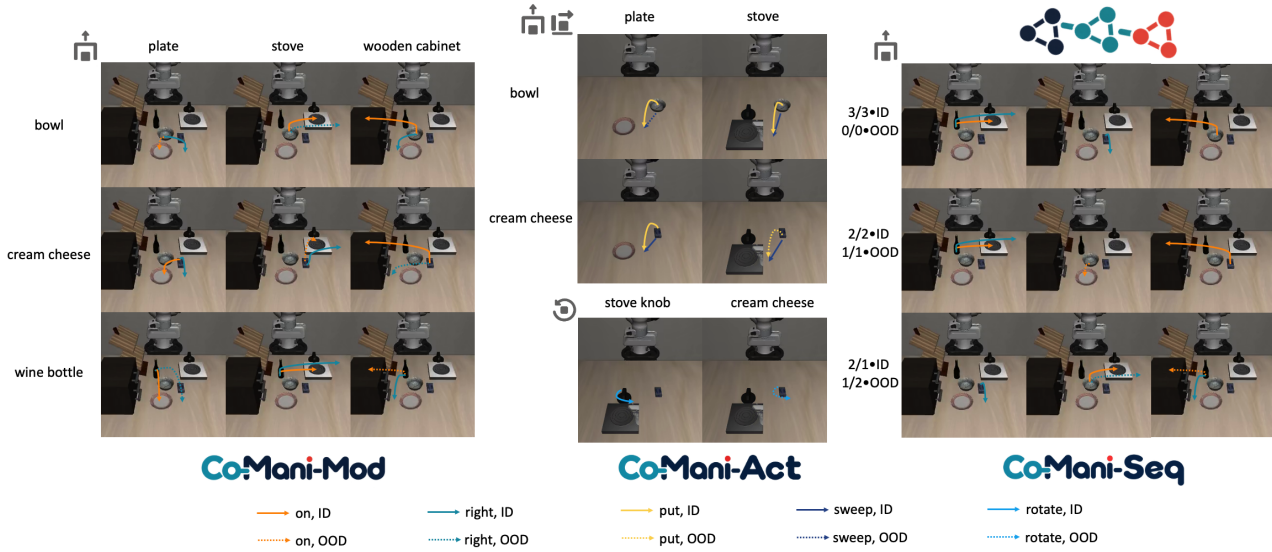


Fig. 2. CoMani task construction and splits. Left: CoMani-Mod varies modifiers with a fixed put action. Center: CoMani-Act varies action types with a fixed right modifier. Right: CoMani-Seq composes atomic tasks from CoMani-Mod into six three-step sequences, shown in three rows with execution proceeding from left to right.

TABLE I
CoMANI TASK LISTS AND ID/OOD SPLITS. IN (C), ID/OOD GIVES ATOMIC TASK COUNTS; ATOMIC IDs REFER TO (A), WITH OOD IDs UNDERLINED.

(a) CoMani-Mod			(b) CoMani-Act		
ID	Task instruction	Split	ID	Task instruction	Split
0	Put the bowl on the stove.	ID	0	Put the bowl to the right of the plate.	ID
1	Put the bowl on the wooden cabinet.	ID	1	Put the bowl to the right of the stove.	ID
2	Put the bowl to the right of the plate.	ID	2	Put the cream cheese to the right of the plate.	ID
3	Put the bowl to the right of the wooden cabinet.	ID	3	Rotate the stove knob right.	ID
4	Put the cream cheese on the plate.	ID	4	Sweep the bowl to the right of the stove.	ID
5	Put the cream cheese on the wooden cabinet.	ID	5	Sweep the cream cheese to the right of the plate.	ID
6	Put the cream cheese to the right of the plate.	ID	6	Sweep the cream cheese to the right of the stove.	ID
7	Put the cream cheese to the right of the stove.	ID	7	Put the cream cheese to the right of the stove.	OOD
8	Put the wine bottle on the plate.	ID	8	Rotate the cream cheese right.	OOD
9	Put the wine bottle on the stove.	ID	9	Sweep the bowl to the right of the plate.	OOD
10	Put the wine bottle to the right of the stove.	ID	(c) CoMani-Seq		
11	Put the wine bottle to the right of the wooden cabinet.	ID	ID	ID/OOD	Sequence
12	Put the bowl on the plate.	OOD	0	3/0	Wine bottle on stove → cream cheese right of plate → bowl on wooden cabinet
13	Put the bowl to the right of the stove.	OOD	1	3/0	Wine bottle right of stove → cream cheese right of plate → bowl on wooden cabinet
14	Put the cream cheese on the stove.	OOD	2	2/1	Wine bottle on stove → bowl on plate → cream cheese on wooden cabinet
15	Put the cream cheese to the right of the wooden cabinet.	OOD	3	2/1	Wine bottle right of stove → bowl on plate → cream cheese on wooden cabinet
16	Put the wine bottle on the wooden cabinet.	OOD	4	2/1	Cream cheese right of plate → bowl on stove → wine bottle on wooden cabinet
17	Put the wine bottle to the right of the plate.	OOD	5	1/2	Cream cheese right of plate → bowl right of stove → wine bottle on wooden cabinet
					Atomic IDs
					9 → 6 → 1
					10 → 6 → 1
					9 → <u>12</u> → 5
					10 → <u>12</u> → 5
					6 → 0 → <u>16</u>
					6 → <u>13</u> → <u>16</u>

respectively. The final pair tests the effect of replacing an ID atomic task with its OOD modifier variant while keeping the remaining subtasks and their order fixed. We compare environment-based switching (ES) with self-switching (SS), where GraphPoint triggers subtask transitions using its own predicted progress. The standard progress head uses patient

and target summaries (P+T). GraphPoint achieves 51.67% SR and 79.44% PR with environment-based switching, and 38.33% SR and 78.33% PR with self-switching. Thus, self-switching reduces SR by 13.33 percentage points while PR decreases by 1.11 points, indicating that similar partial progress does not guarantee completion of the full sequence.

TABLE II
SUCCESS RATES (%) ON CoMANI-MOD. TASK IDS FOLLOW TABLE I.

Method	Mean SR		ID SR												OOD SR					
	ID	OOD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
PointBridge	3.3														20.0	0.0	0.0	0.0	0.0	0.0
PointPolicy	21.4	15.0	3.3	36.7	66.7	50.0	0.0	0.0	16.7	0.0	23.3	0.0	13.3	46.7	10.0	0.0	0.0	6.7	0.0	73.3
DP3	73.6	57.2	73.3	100.0	100.0	100.0	53.3	100.0	0.0	80.0	80.0	86.7	26.7	83.3	86.7	63.3	60.0	93.3	16.7	23.3
ACT	20.0	7.2	33.3	20.0	6.7	23.3	13.3	56.7	10.0	3.3	63.3	0.0	0.0	10.0	30.0	0.0	0.0	13.3	0.0	0.0
DP	40.0	10.0	43.3	36.7	16.7	26.7	50.0	66.7	20.0	33.3	80.0	16.7	36.7	53.3	30.0	0.0	0.0	30.0	0.0	0.0
$\pi_{0.5}$	91.1	48.3	100.0	100.0	56.7	100.0	73.3	100.0	96.7	83.3	100.0	100.0	83.3	100.0	80.0	33.3	6.7	80.0	6.7	83.3
GraphPoint	96.4	75.6	100.0	100.0	100.0	86.7	93.3	100.0	100.0	80.0	100.0	100.0	96.7	100.0	100.0	63.3	73.3	76.7	73.3	66.7

TABLE III
SUCCESS RATES (%) ON CoMANI-ACT. TASK IDS FOLLOW TABLE I.

Method	Mean SR		ID SR							OOD SR		
	ID	OOD	0	1	2	3	4	5	6	7	8	9
DP3	83.3	60.0	80.0	93.3	96.7	100.0	100.0	86.7	26.7	80.0	0.0	100.0
$\pi_{0.5}$												
GraphPoint	90.5	64.4	50.0	100.0	83.3	100.0	100.0	100.0	100.0	26.7	76.7	90.0

DP3 obtains 0% SR and 11.11% PR even with environment-based switching. The $\pi_{0.5}$ evaluation remains pending.

VI. CONCLUSION

REFERENCES

D. Which Components Matter?

Table V examines GraphPoint variants on CoMani-Mod. We remove role-aware point collapse (RPC), replace chain attention with full connectivity, merge the entity graph into a single point set, remove current-TCP centering, or predict Cartesian delta actions instead of point trajectories.

Geometry and graph structure. Removing RPC reduces OOD success by 31.1 percentage points, compared with a 5.8-point ID decrease, consistent with its intended role in reducing dependence on detailed training geometry. Full graph connectivity retains 95.28% ID success but lowers OOD success to 63.89%, supporting patient-mediated interaction as a useful inductive bias. Merging all entity points yields 27.78% OOD success despite retaining 12 summary tokens in total. This variant jointly removes role embeddings, separate local entity groups, and chain connectivity, and reads shared summaries for progress; it therefore tests the structured encoder as a whole rather than any single component.

Coordinate and output representations. Removing current-TCP centering lowers OOD success to 25.00% while retaining 92.50% ID success, indicating that relative geometry is particularly useful for held-out combinations. Replacing point trajectories with Cartesian delta actions gives 95.00% ID and 73.33% OOD success, close to the full model. The observed gains are therefore more pronounced for relative coordinates, geometric augmentation, and role structure than for point output alone. These comparisons describe the reported checkpoint evaluations; repeated-training estimates are needed to assess the reliability of small differences.

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proceedings of Robotics: Science and Systems*, 2023. [Online]. Available: <https://roboticsproceedings.org/rss19/p016.html>
- [2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor policy learning via action diffusion," in *Proceedings of Robotics: Science and Systems*, 2023. [Online]. Available: <https://diffusion-policy.cs.columbia.edu/>
- [3] Physical Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, " $\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [4] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D Diffusion Policy: Generalizable visuomotor policy learning via simple 3D representations," in *Proceedings of Robotics: Science and Systems*, 2024. [Online]. Available: <https://3d-diffusion-policy.github.io/>
- [5] S. Haldar and L. Pinto, "Point Policy: Unifying observations and actions with key points for robot manipulation," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 1825–1846. [Online]. Available: <https://proceedings.mlr.press/v305/haldar25a.html>
- [6] S. Haldar, L. Johannsmeier, L. Pinto, A. Gupta, D. Fox, Y. Narang, and A. Mandlekar, "Point Bridge: 3D representations for cross domain policy learning," *arXiv preprint arXiv:2601.16212*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.16212>
- [7] Y. Zhu, Z. Jiang, P. Stone, and Y. Zhu, "Learning generalizable manipulation policies with object-centric 3D representations," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3418–3433. [Online]. Available: <https://proceedings.mlr.press/v229/zhu23b.html>
- [8] G. Chen, T. Cui, M. Wang, C. Yang, M. Hu, H. Lu, Y. Mu, Z. Peng, T. Zhou, X. Jiang, Y. Yang, and Y. Yue, "GraphMimic: Graph-to-graphs generative modeling from videos for policy learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 1756–1768. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/html/Chen_GraphMimic_Graph-to-Graphs_Generative_Modeling_from_Videos_for_Policy_Learning_CVPR_2025_paper.html

TABLE IV

CoMANI-SEQ SUCCESS RATE (SR) AND PROGRESS RATE (PR), IN %. ES/SS USE ENVIRONMENT/PREDICTED-PROGRESS SIGNALS FOR SWITCHING.
MEAN GROUPS BY OOD COUNT; TASK IDS FOLLOW TABLE I.

Method	Switch	Mean								3 ID				2 ID + 1 OOD				2/1 ID + 1/2 OOD			
		All		0 OOD		1 OOD		2 OOD		0		1		2		3		4		5	
		SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR
DP3	ES	0.0	11.1	0.0	16.7	0.0	11.1	0.0	0.0	0.0	26.7	0.0	6.7	0.0	30.0	0.0	3.3	0.0	0.0	0.0	0.0
$\pi_{0.5}$	ES																				
GraphPoint	ES	51.7	79.4	75.0	95.0	50.0	81.1	10.0	43.3	60.0	90.0	90.0	100.0	30.0	76.7	30.0	70.0	90.0	96.7	10.0	43.3
GraphPoint	SS	38.3	78.3	65.0	90.0	26.7	77.8	20.0	56.7	90.0	96.7	40.0	83.3	20.0	76.7	10.0	73.3	50.0	83.3	20.0	56.7

TABLE V

CoMANI-MOD ABLATIONS: SUCCESS RATES (%). TASK IDS FOLLOW TABLE I.

Variant	Mean SR		ID SR												OOD SR					
	ID	OOD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
w/o RPC	90.6	44.4	100.0	96.7	100.0	100.0	66.7	96.7	96.7	83.3	93.3	100.0	70.0	83.3	20.0	90.0	80.0	6.7	10.0	60.0
Full connectivity	95.3	63.9	96.7	96.7	100.0	100.0	100.0	83.3	100.0	86.7	100.0	100.0	80.0	100.0	66.7	86.7	70.0	50.0	36.7	73.3
Merged entity point set	82.8	27.8	93.3	96.7	100.0	100.0	93.3	26.7	83.3	56.7	100.0	96.7	50.0	96.7	60.0	0.0	36.7	3.3	3.3	63.3
w/o TCP centering	92.5	25.0	90.0	100.0	100.0	96.7	86.7	100.0	96.7	46.7	100.0	100.0	93.3	100.0	3.3	63.3	3.3	6.7	6.7	66.7
Delta-action output	95.0	73.3	96.7	100.0	100.0	100.0	100.0	96.7	80.0	86.7	100.0	100.0	80.0	100.0	96.7	86.7	76.7	76.7	50.0	53.3
Full GraphPoint	96.4	75.6	100.0	100.0	100.0	86.7	93.3	100.0	100.0	80.0	100.0	100.0	96.7	100.0	100.0	63.3	73.3	76.7	73.3	66.7

- [9] H. Qi, C. Chen, and H. Yang, “Compose by Focus: Scene graph-based atomic skills,” in *IEEE International Conference on Robotics and Automation*, 2026. [Online]. Available: <https://arxiv.org/abs/2509.16053>
- [10] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 894–906. [Online]. Available: <https://proceedings.mlr.press/v164/shridhar22a.html>
- [11] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: Robot manipulation with multimodal prompts,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 14975–15022. [Online]. Available: <https://proceedings.mlr.press/v202/jiang23b.html>
- [12] J. A. Mendez, M. Hussing, M. Gummadi, and E. Eaton, “CompoSuite: A compositional reinforcement learning benchmark,” in *Proceedings of The 1st Conference on Lifelong Learning Agents*, ser. Proceedings of Machine Learning Research, vol. 199. PMLR, 2022, pp. 982–1003. [Online]. Available: <https://proceedings.mlr.press/v199/mendez22a.html>
- [13] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310>
- [14] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022. [Online]. Available: <https://arxiv.org/abs/2112.03227>
- [15] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, “C-Pack: Packed resources for general chinese embeddings,” in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. [Online]. Available: <https://arxiv.org/abs/2309.07597>
- [16] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/45c1f6a8cbf2da59ebf2c802b4f742cd-Paper-Conference.pdf
- [17] S. Jung, A. Zholus, M. Sundermeyer, C. Doersch, R. Goroshin, D. J. Tan, S. Chandar, R. Triebel, and F. Tombari, “TAPNext++: What’s next for tracking any point (TAP)?” *arXiv preprint arXiv:2604.10582*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.10582>
- [18] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [19] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 1820–1864. [Online]. Available: <https://proceedings.mlr.press/v229/mandlekar23a.html>